



µLab worksheet • Kubernetes in one container

Group _____ Date _____ Time _____

Driver (types) _____ Mapper (looks up) _____ Referee (demos) _____

Hand this in at the end of the round. The referee presents this work in 90 seconds at the caucus, so the referee has to follow it, not just watch it.

1 • START THE CLUSTER, THEN GET A CLIENT • STEPS 1 AND 2

1 ☐ Create a container that is a Kubernetes cluster

```
mkdir kube
docker run -d --name k8s --privileged --tmpfs /run --tmpfs /var/run \
  -p 6443:6443 -v "$PWD/kube:/output" \
  rancher/k3s:v1.36.4-k3s1 server --disable=traefik \
  --write-kubeconfig /output/config --write-kubeconfig-mode 644
```

`-d` Detached. It runs in the background and you get your prompt back.
`--name k8s` A name of your choosing. Without it Docker still identifies the container, but by its id: 8d7726f96f747554440565a92a034cda1f767726619081595bab18cdf76987. Every later command would have to carry that, or its short form 8d7726f96f74.

`--privileged` Lifts the usual restrictions. k3s mounts cgroups and runs **its own** container runtime inside this container.
`--tmpfs /run` Two small writable in-memory filesystems k3s needs (also `/var/run`).
`-p 6443:6443` **HOST:CONTAINER.** Port 6443 on your laptop is forwarded to 6443 inside the container. 6443 is where the cluster's **API server** listens, and step 2 is going to talk to it.

`-v "$PWD/kube:/output"` Bind-mounts your new `kube` folder into the container, so a file written there lands on your laptop.
`rancher/k3s:v1.36.4-k3s1` The image. `rancher/k3s` is the repository, `v1.36.4-k3s1` the tag. You named no registry, so Docker went to **Docker Hub**.

`server` **Not a docker flag.** Everything after the image name is the command handed to the program inside. `server` tells k3s to be the cluster.

`--disable=traefik` Also k3s's. Traefik is an **ingress controller**: it routes HTTP arriving from outside to services inside. It claims ports 80 and 443 and adds startup time; nothing today arrives from outside.

`--write-kubeconfig` Writes the cluster's address and credentials to `/output/config`, which is your `kube/config`. `-mode 644` makes it readable without `sudo`.

After a few seconds `kube/config` exists on your laptop. Open it. One line reads `server: https://127.0.0.1:6443` — that is the only address anything will need.

2 ☐ Get a kubectl, and point it at that address

`kubectl` is **not part of the cluster**. It is an ordinary program that sends HTTPS requests to the address above, so it can live anywhere. Pick one.

(a) On Linux — install it on your laptop

```
sudo snap install kubectl --classic
export KUBECONFIG=$PWD/kube/config
```

It reaches the cluster through the port you published in step 1. This is exactly how you would drive a real cluster from your own machine.

(b) Anywhere, including macOS and Windows — run kubectl in its own container

```
docker run --rm -it --network container:k8s \
  -v "$PWD/kube:/kube:ro" -e KUBECONFIG=/kube/config \
  --entrypoint sh alpine/kubectl
```

`--rm` Delete this container the moment you leave it. The cluster in step 1 used `-d` because it must outlive your prompt; this one is disposable and you can start another whenever you like.

`-it` `-i` keeps the input open, `-t` gives a terminal. Together: an interactive shell instead of a single command.
`--network container:k8s` Do not give this container its own network — **join the one the cluster container already has**. That is why `127.0.0.1:6443` works in here: it is the cluster's own loopback, which is exactly the address written in `kube/config`.

`-v "$PWD/kube:/kube:ro"` Bind-mount again, the other way round: your `kube` folder appears inside this container at `/kube`. `:ro` makes it **read-only** — a client has no business editing the cluster's credentials.

```
-e KUBECONFIG=/kube/config  Sets an environment variable inside the container. kubectl reads KUBECONFIG to find out which cluster
                             to talk to. Note the path is the one inside this container, not the one on your laptop.
--entrypoint sh              Overrides what the image runs by default. alpine/kubectl is built to run kubectl and nothing else, so
                             without this you would get one command per docker run. Swapping in a shell lets you type many.
alpine/kubectl              The image: kubectl and a small Alpine base, nothing else. No registry named, so Docker Hub again.
```

From this prompt, commands are typed as plain `kubectl`, with no `docker` in front.

There are now **two containers running**, and a second terminal shows both:

```
docker ps
NAME          IMAGE              STATUS
client       alpine/kubectl    Up 2 seconds
k8s          rancher/k3s:v1.36.4-k3s1  Up 4 minutes
```

One is the cluster, one is the client. Leaving the client stops only the client: `--rm` deletes it, and the cluster carries on without it.

Either way, you are outside the cluster.

Nothing from here on is typed **inside** the cluster. The cluster is one program and the client is another, and the only thing joining them is the address in `kube/config`. Which did you pick, (a) or (b)? _____

Now check that it reaches the cluster.

`kubectl get nodes` asks the API server for the machines that make up the cluster. There will be exactly one: the container you started in step 1. If this prints a table, your client found the cluster and everything after this will work.

```
kubectl get nodes
```

Expect something close to:

NAME	STATUS	ROLES	AGE	VERSION
7c19eb7874a1	Ready	control-plane	11s	v1.36.4+k3s1

Instead got `The connection to the server localhost:8080 was refused?` Then `kubectl` never found the config and fell back to a built-in default. Check that `kube/config` exists and is not empty, and that you are running from the folder that contains `kube`.

2 • SOME DEFINITIONS

Step 1 produced a Kubernetes cluster inside one container. Step 2 produced a `kubectl` outside it that can reach the cluster's API. Steps 3 to 6 use the words below.

- kubectl** The cluster's command-line client. It runs nothing itself: every command becomes an authenticated HTTPS request to the cluster's API, and the reply is printed.
On an HPC cluster running **Slurm**, work is submitted by first logging in to a **login node** and running `sbatch` there. Kubernetes has no login node. `kubectl` carries the credentials from `kube/config` and authenticates each request on its own, so the machine it runs on is doing the login node's job.
- pod** **A running container, as Kubernetes counts it.** `docker run` produces a container; Kubernetes produces a pod, with the container inside it. A pod may hold several containers that have to share one machine and one IP address, but holding a single container is the ordinary case. **Pods are never created directly.**
- Deployment** **A written request, stored by the cluster,** of the form *keep N pods of image X running*. Creating one starts no container: the request is sent, the cluster saves it, and the command returns. The pods appear a moment later, started by something else. Why request it rather than start it? On a real cluster there are many machines, and picking one by hand would mean tracking which has room. Worse, machines and processes fail: a container started by hand stays dead, while a request that is stored can go on being satisfied after the failure.
- controller** **A program inside the cluster that reads the stored requests and changes the cluster until they are met.** Take a Deployment that says three pods. The controller counts the pods that exist. Two, so it starts one. Four, and it would stop one. Then it counts again, and it keeps counting for as long as the request is stored.
This is why a deleted pod comes back: deleting a pod changes the count, not the request.
- Job** **A different stored request: run image X once, until it finishes.** Its controller starts one pod, waits for a successful exit, and stops. Nothing is replaced.

3 • DRIVE THE CLUSTER • STEPS 3 TO 6

Type these with the `kubectl` set up in step 2.

3 ☐ Create a Deployment

The aim of this round is to watch what a cluster does when a container stops. So the object to create is a Deployment: a stored request for three pods that should be running. Three rather than one, only so that there is more to watch.

Read step 4 first. What happens next happens in seconds, and step 4 is how to see it. Nothing is lost by being slow, but the first few seconds are the interesting ones.

```
kubectl create deployment whale --image=ghcr.io/oesteban/whalesay --replicas=3
```

create The verb: make a new object. Others are `get`, `describe`, `scale`, `delete`, `logs`.
deployment The **kind** of object to create. `job`, `pod`, `service` are other kinds.
whale The **name**, chosen freely. Every later command refers to it by this name. It also becomes the label `app=whale` and the prefix of every pod name it creates, which is why the pods come out called `whale-544548b554-kpnfx`.
--image=... Which image the pods run. The only part the cluster has to fetch from a registry.
--replicas=3 How many pods the request asks for. Leave it out and the default is 1.

Why `ghcr.io/...` spelled out, when step 1 named no registry at all? There is a second copy of this image on Docker Hub, and it does not work on every laptop. Panel 12, the annex, has the answer and a command to check it.

4 ☐ Watch what the cluster does with it

`kubectl get pods` prints the pods once and returns. Adding `--watch` keeps it open instead, printing **one more line every time something changes**. It is not a refreshing table: the lines pile up, oldest at the top, and that record is the point.

```
kubectl get pods --watch
```

get List objects, one line each. No name given, so it lists them all.
pods The kind to list. `pod`, `pods` and the short `po` all work. `kubectl get deployments` and `kubectl get jobs` list the other two kinds.
--watch Do not exit; report every change as it happens. `-w` is the short form. **Ctrl-C** ends it.

Within about a minute the output looks like this:

NAME	READY	STATUS	RESTARTS
whale-544548b554-dkrkr	0/1	ContainerCreating	0
whale-544548b554-dkrkr	1/1	Running	0
whale-544548b554-dkrkr	0/1	Completed	0
whale-544548b554-dkrkr	1/1	Running	1 (2s ago)
whale-544548b554-dkrkr	0/1	Completed	1 (2s ago)
whale-544548b554-dkrkr	0/1	CrashLoopBackOff	1 (11s ago)
whale-544548b554-dkrkr	1/1	Running	2 (12s ago)

One pod's history, read downwards: created, ran, finished, ran again, finished again. **RESTARTS** counts how many times the container has been started again, and it keeps climbing. Record an early line and a late one in panel 5, then **Ctrl-C**.

To run it again from the beginning: delete the Deployment and repeat step 3, with `--watch` started first in a second client.

```
kubectl delete deployment whale
```

`delete` removes the stored request, and its pods go with it.

5 ☐ Create a Job from the same image

A Job is the other kind of stored request: run the image once, until it finishes. The image and the cluster are the same as in step 3, and only the kind is different — so anything that happens differently is caused by the kind alone.

```
kubectl create job whale --image=ghcr.io/oesteban/whalesay -- cowsay 'a cluster ran me'
```

job The kind. Everything else on this line works exactly as it did in step 3.
-- **Ends kubectl's own arguments.** What follows is not for kubectl: it is the command to run inside the container, and it replaces the one built into the image. The same idea as `--entrypoint` in step 2, and of `ENTRYPOINT` in a Dockerfile.
cowsay 'a cluster ran me' That command and its single argument. `cowsay` is the program inside the image; the quoted text is what it prints.

Then `kubectl get pods` once more. The Job's pod is the one whose name carries a single hash rather than two.

6 ☐ Read what the Job printed

The pod ran `cowsay` and exited. What it printed is still held by the cluster, and this is how to see it.

```
kubectl logs job/whale
```

logs Print what a container wrote to its output. Containers have no screen; this is the screen.
job/whale **KIND/NAME.** A Job does not have logs — its **pod** does. Naming the Job is enough: `kubectl` finds the pod it made. `deploy/whale` works the same way, and a bare pod name also works.

4 • WHAT YOU STARTED

From `kubectl get nodes`:

NAME	VERSION
STATUS	AGE when first Ready

5 • TWO LINES FROM THE WATCH, AND THE REASON

An early line

STATUS _____

RESTARTS _____

A later line

STATUS _____

RESTARTS _____

Run `docker ps` on your laptop: compare CONTAINER ID with the NAME in panel 4.

The container did not crash. It printed its whale and exited **successfully**. So why is Kubernetes starting it again?

The Job in step 5 used **the same image** and did not do this. What did you change, in one sentence?

If STATUS reads **CrashLoopBackOff** : that is the status shown while a controller waits before starting a stopped container again. The wait grows — 10s, 20s, 40s, doubling to a five-minute ceiling. It does not mean the container crashed.

Then read what Kubernetes actually claims.

`kubectl describe pod <one of them>` and find the block called **Last State**.

Exit Code _____ Reason _____

Now read the message beside **State: Waiting**. It contains the words `restarting failed container`.

Was it a failure? Reconcile those two lines, in one sentence.

6 • WHAT THE LOGS WILL TELL YOU

Logs are the first place you look when something is wrong, and they are not only for things that are wrong. Try all four, inside the cluster.

`kubectl logs deploy/whale`

There are three pods. Which one did it pick, and how do you know?

`kubectl logs <pod> --previous`

The container that **already died**. Why would that ever be the only copy left?

`kubectl logs -l app=whale --prefix`

All three at once. What does the prefix tell you?

`kubectl logs <pod> --timestamps`

Now you can see **when**. How far apart are the restarts?

A pod is gone and you want to know why. Which of those four do you reach for, and what would you have lost if you had waited an hour?

Reached this line? Then you have solved the challenge.



Referee: tick the box on the left now, and write the time. Do not wait for the caucus. Then carry on below — the extensions are not optional, they are the rest of the round.

Solved at _____ Referee's initials _____

7 • NOW USE THE IMAGE YOU BUILT ON FRIDAY

Everything so far ran **someone else's** image. Repeat step 2 with your own, the one your GitHub Actions workflow published on Friday:

`kubectl create deployment mine --image=ghcr.io/<your-handle>/whalesay`

Your handle _____ Did it pull? _____

If it did not, `kubectl describe pod` will name the reason. Write it down, then fix it.

Nobody told the cluster where `ghcr.io` is, and nobody gave it a password. So: what **is** a registry, and what had to be true of your package for this to work at all?

On Friday you pushed to **GHCR**. When you type `docker pull alpine` you get an image from **Docker Hub**, and you never type a registry name. Where did that default come from, and what is the full name your `alpine` actually has?

8 • THE CONTROLLER'S OWN ACCOUNT

In panel 5 you read **Last State** from `kubectl describe pod`. The same output ends with an **Events** block. Read it, then widen to the whole cluster:

```
kubectl get events --sort-by=.lastTimestamp | tail -8
```

`events` is a kind of object like `pods`: the cluster records one every time it decides something. `--sort-by` takes a field of the object, oldest first, so `tail -8` leaves the eight most recent.

Nobody wrote this log. Who did, and what is it a record of — what you asked for, or what the cluster did about it?

9 • TWO IMAGE STORES, ONE NAME

Inside the cluster: `crictl images`.

On your laptop, in another terminal: `docker images`.

whalesay in `crictl`? _____ in `docker`? _____

Nobody copied anything from your laptop into the cluster. So how did the image get there?

10 • A WISH IS NOT A COMMAND

`kubectl scale deployment whale --replicas=5`, wait, then `kubectl scale deployment whale --replicas=0`. Count the pods after each.

`scale` edits one field of a request that already exists — here `replicas` — rather than creating anything. `deployment whale` is KIND then NAME, the long form of `deploy/whale`.

pods at 5 _____ pods at 0 _____

Now delete the **Job's** pod: `kubectl delete pod <the job pod>`. Does it come back? Does a Deployment's pod come back? Why are the answers different?

11 • EXTRA MILE • SATISFY THE DEPLOYMENT INSTEAD OF ABANDONING IT

In step 5 you stopped the restarting by changing the **object**. There is a second way that leaves it a Deployment: same image, no rebuild, no re-push, sitting at `RESTARTS 0` and `Running`. Write the command.

You have now satisfied a Deployment two ways. Which would you use for a web server, and which for a nightly report?

No hint. The image runs one program and then stops; nothing says it has to.

12 • ANNEX • WHY THE IMAGE COMES FROM GHCR.IO AND NOT DOCKER HUB

Two copies of this image exist under the same short name, and they are not the same thing. Run these two *on the laptop*, not in the `kubectl` container:

```
docker buildx imagetools inspect ghcr.io/oesteban/whalesay
docker buildx imagetools inspect oesteban/whalesay
```

The first prints a `Manifests:` list with `Platform: linux/amd64` **and** `Platform: linux/arm64`. The second prints no list at all: one image, `linux/amd64` only.

An Apple Silicon Mac runs an **arm64** Linux VM, and the `containerd` inside `k3s` has no emulation to fall back on. The Docker Hub copy would fail there with `exec format error` and the pod would never start. The GHCR copy was built for both architectures by GitHub Actions on Friday.

On Linux or Windows, on Intel or AMD, the Docker Hub copy is fine. Check the manifests above, then run it alongside:

```
kubectl create deployment hub --image=oesteban/whalesay --replicas=1
```

Which architecture is this laptop? _____

Done when the referee can say, without reading this sheet, why the same image restarts forever as a Deployment and finishes as a Job.